

✓ Lab 4: Data Imputation using an Autoencoder

In this lab, you will build and train an autoencoder to impute (or "fill in") missing data.

We will be using the Adult Data Set provided by the UCI Machine Learning Repository [1], available at <https://archive.ics.uci.edu/ml/datasets/adult>. The data set contains census record files of adults, including their age, marital status, the type of work they do, and other features.

Normally, people use this data set to build a supervised classification model to classify whether a person is a high income earner. We will not use the dataset for this original intended purpose.

Instead, we will perform the task of imputing (or "filling in") missing values in the dataset. For example, we may be missing one person's marital status, and another person's age, and a third person's level of education. Our model will predict the missing features based on the information that we do have about each person.

We will use a variation of a denoising autoencoder to solve this data imputation problem. Our autoencoder will be trained using inputs that have one categorical feature artificially removed, and the goal of the autoencoder is to correctly reconstruct all features, including the one removed from the input.

In the process, you are expected to learn to:

1. Clean and process continuous and categorical data for machine learning.
2. Implement an autoencoder that takes continuous and categorical (one-hot) inputs.
3. Tune the hyperparameters of an autoencoder.
4. Use baseline models to help interpret model performance.

[1] Dua, D. and Karra Taniskidou, E. (2017). UCI Machine Learning Repository [<http://archive.ics.uci.edu/ml>]. Irvine, CA: University of California, School of Information and Computer Science.

What to submit

Submit a PDF file containing all your code, outputs, and write-up. You can produce a PDF of your Google Colab file by going to File > Print and then save as PDF. The Colab instructions have more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

✓ Colab Link

Include a link to your Colab file here. If you would like the TA to look at your Colab file in case your solutions are cut off, **please make sure that your Colab file is publicly accessible at the time of submission.**

Colab Link:

```
import csv
import numpy as np
import random
import torch
import torch.utils.data
```

✓ Part 0

We will be using a package called `pandas` for this assignment.

If you are using Colab, `pandas` should already be available. If you are using your own computer, installation instructions for `pandas` are available here: <https://pandas.pydata.org/pandas-docs/stable/install.html>

```
import pandas as pd
```

✓ Part 1. Data Cleaning [15 pt]

The adult.data file is available at (<https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data>)

The function `pd.read_csv` loads the adult.data file into a pandas dataframe. You can read about the pandas documentation for `pd.read_csv` at https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

```
header = ['age', 'work', 'fnlwgt', 'edu', 'yrelu', 'marriage', 'occupation',
          'relationship', 'race', 'sex', 'capgain', 'caploss', 'workhr', 'country']
df = pd.read_csv(
    "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data",
    names=header,
    index_col=False)
```

```
/tmp/ipykernel_441/1831985018.py:3: ParserWarning: Length of header or names does not match length of data. This lead
df = pd.read_csv(
```

```
df.shape # there are 32561 rows (records) in the data frame, and 14 columns (features)
```

```
(32561, 14)
```

▼ Part (a) Continuous Features [3 pt]

For each of the columns `["age", "yrelu", "capgain", "caploss", "workhr"]`, report the minimum, maximum, and average value across the dataset.

Then, normalize each of the features `["age", "yrelu", "capgain", "caploss", "workhr"]` so that their values are always between 0 and 1. Make sure that you are actually modifying the dataframe `df`.

Like numpy arrays and torch tensors, pandas data frames can be sliced. For example, we can display the first 3 rows of the data frame (3 records) below.

```
df[:3] # show the first 3 records
```

	age	work	fnlwgt	edu	yrelu	marriage	occupation	relationship	race	sex	capgain	caploss	workhr	country
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States

Alternatively, we can slice based on column names, for example `df["race"]`, `df["hr"]`, or even index multiple columns like below.

```
subdf = df[["age", "yrelu", "capgain", "caploss", "workhr"]]
subdf[:3] # show the first 3 records
```

	age	yrelu	capgain	caploss	workhr
0	39	13	2174	0	40
1	50	13	0	0	13
2	38	9	0	0	40

Numpy works nicely with pandas, like below:

```
np.sum(subdf["caploss"])
```

```
np.int64(2842700)
```

Just like numpy arrays, you can modify entire columns of data rather than one scalar element at a time. For example, the code

```
df["age"] = df["age"] + 1
```

would increment everyone's age by 1.

```

for col in subdf:
    print(f"{col}:")
    print("  min =", df[col].min())
    print("  max =", df[col].max())
    print("  mean =", df[col].mean())

for col in subdf:
    min_val = df[col].min()
    max_val = df[col].max()
    df[col] = (df[col] - min_val) / (max_val - min_val)

```

```

age:
  min = 17
  max = 90
  mean = 38.58164675532078
yrelu:
  min = 1
  max = 16
  mean = 10.0806793403151
capgain:
  min = 0
  max = 99999
  mean = 1077.6488437087312
caploss:
  min = 0
  max = 4356
  mean = 87.303829734959
workhr:
  min = 1
  max = 99
  mean = 40.437455852092995

```

Part (b) Categorical Features [1 pt]

What percentage of people in our data set are male? Note that the data labels all have an unfortunate space in the beginning, e.g. " Male" instead of "Male".

What percentage of people in our data set are female?

```

# hint: you can do something like this in pandas
male = sum(df["sex"] == " Male")
female = sum(df["sex"] == " Female")
total = male+female
female_percent = female/(total) *100
print(f"female_percent: .2f}%")

```

33.08%

Part (c) [2 pt]

Before proceeding, we will modify our data frame in a couple more ways:

1. We will restrict ourselves to using a subset of the features (to simplify our autoencoder)
2. We will remove any records (rows) already containing missing values, and store them in a second dataframe. We will only use records without missing values to train our autoencoder.

Both of these steps are done for you, below.

How many records contained missing features? What percentage of records were removed?

```

contcols = ["age", "yrelu", "capgain", "caploss", "workhr"]
catcols = ["work", "marriage", "occupation", "edu", "relationship", "sex"]
features = contcols + catcols
df = df[features]

```

```

missing = pd.concat([df[c] == "?" for c in catcols], axis=1).any(axis=1)
df_with_missing = df[missing]
df_not_missing = df[~missing]

remove = len(df_with_missing)/(len(df_with_missing)+len(df_not_missing)) *100
num_missing = len(df_with_missing)

```

```
print("Missing records:", num_missing)
print(f"{{remove:.2f}} %")
```

```
Missing records: 1843
5.66 %
```

Part (d) One-Hot Encoding [1 pt]

What are all the possible values of the feature "work" in `df_not_missing`? You may find the Python function `set` useful.

```
set(df_not_missing["work"])
```

```
{' Federal-gov',
 ' Local-gov',
 ' Private',
 ' Self-emp-inc',
 ' Self-emp-not-inc',
 ' State-gov',
 ' Without-pay'}
```

We will be using a one-hot encoding to represent each of the categorical variables. Our autoencoder will be trained using these one-hot encodings.

We will use the pandas function `get_dummies` to produce one-hot encodings for all of the categorical variables in `df_not_missing`.

```
data = pd.get_dummies(df_not_missing)
```

```
data[:3]
```

	age	yredu	capgain	caploss	workhr	work_Federal-gov	work_Local-gov	work_Private	work_Self-emp-inc	work_Self-emp-not-inc	...	edu_Prof-school	edu_Some-college	relationsh_Husb
0	0.301370	0.800000	0.02174	0.0	0.397959	False	False	False	False	False	...	False	False	F
1	0.452055	0.800000	0.00000	0.0	0.122449	False	False	False	False	True	...	False	False	.
2	0.287671	0.533333	0.00000	0.0	0.397959	False	False	True	False	False	...	False	False	F

3 rows x 57 columns

Part (e) One-Hot Encoding [2 pt]

The dataframe `data` contains the cleaned and normalized data that we will use to train our denoising autoencoder.

How many **columns** (features) are in the dataframe `data`?

Briefly explain where that number come from.

```
features = data.shape[1]
print(features)
```

```
57
```

Part (f) One-Hot Conversion [3 pt]

We will convert the pandas data frame `data` into numpy, so that it can be further converted into a PyTorch tensor. However, in doing so, we lose the column label information that a panda data frame automatically stores.

Complete the function `get_categorical_value` that will return the named value of a feature given a one-hot embedding. You may find the global variables `cat_index` and `cat_values` useful. (Display them and figure out what they are first.)

We will need this function in the next part of the lab to interpret our autoencoder outputs. So, the input to our function `get_categorical_values` might not actually be "one-hot" -- the input may instead contain real-valued predictions from our neural network.

```
data_np = data.values.astype(np.float32)
```

```
cat_index = {} # Mapping of feature -> start index of feature in a record
cat_values = {} # Mapping of feature -> list of categorical values the feature can take

# build up the cat_index and cat_values dictionary
for i, header in enumerate(data.keys()):
    if "_" in header: # categorical header
        feature, value = header.split()
        feature = feature[:-1] # remove the last char; it is always an underscore
        if feature not in cat_index:
            cat_index[feature] = i
            cat_values[feature] = [value]
        else:
            cat_values[feature].append(value)

def get_onehot(record, feature):
    """
    Return the portion of `record` that is the one-hot encoding
    of `feature`. For example, since the feature "work" is stored
    in the indices [5:12] in each record, calling `get_range(record, "work")`
    is equivalent to accessing `record[5:12]`.

    Args:
        - record: a numpy array representing one record, formatted
            the same way as a row in `data_np`
        - feature: a string, should be an element of `catcols`
    """
    start_index = cat_index[feature]
    stop_index = cat_index[feature] + len(cat_values[feature])
    return record[start_index:stop_index]

def get_categorical_value(onehot, feature):
    """
    Return the categorical value name of a feature given
    a one-hot vector representing the feature.

    Args:
        - onehot: a numpy array one-hot representation of the feature
        - feature: a string, should be an element of `catcols`

    Examples:

    >>> get_categorical_value(np.array([0., 0., 0., 0., 0., 1., 0.]), "work")
    'State-gov'
    >>> get_categorical_value(np.array([0.1, 0., 1.1, 0.2, 0., 1., 0.]), "work")
    'Private'
    """
    # <----- TODO: WRITE YOUR CODE HERE ----->
    # You may find the variables `cat_index` and `cat_values`
    # (created above) useful.
    # Find index of largest value (works for one-hot or soft predictions)
    idx = np.argmax(onehot)

    # Map index to actual category name
    return cat_values[feature][idx]
```

```
# more useful code, used during training, that depends on the function
# you write above
```

```
def get_feature(record, feature):
    """
    Return the categorical feature value of a record
    """
    onehot = get_onehot(record, feature)
    return get_categorical_value(onehot, feature)

def get_features(record):
    """
    Return a dictionary of all categorical feature values of a record
    """
    return { f: get_feature(record, f) for f in catcols }
```

▼ Part (g) Train/Test Split [3 pt]

Randomly split the data into approximately 70% training, 15% validation and 15% test.

Report the number of items in your training, validation, and test set.

```
# set the numpy seed for reproducibility
# https://docs.scipy.org/doc/numpy/reference/generated/numpy.random.seed.html
np.random.seed(50)

# todo

# Shuffle indices
num_samples = datanp.shape[0]
indices = np.random.permutation(num_samples)

# Compute split sizes
train_end = int(0.7 * num_samples)
val_end = int(0.85 * num_samples)

# Split indices
train_idx = indices[:train_end]
val_idx = indices[train_end:val_end]
test_idx = indices[val_end:]

# Create datasets
train_data = datanp[train_idx]
val_data = datanp[val_idx]
test_data = datanp[test_idx]

# Report sizes
print("Training set size:", train_data.shape[0])
print("Validation set size:", val_data.shape[0])
print("Test set size:", test_data.shape[0])

Training set size: 21502
Validation set size: 4608
Test set size: 4608
```

✓ Part 2. Model Setup [5 pt]

Part (a) [4 pt]

Design a fully-connected autoencoder by modifying the `encoder` and `decoder` below.

The input to this autoencoder will be the features of the `data`, with one categorical feature recorded as "missing". The output of the autoencoder should be the reconstruction of the same features, but with the missing value filled in.

Note: Do not reduce the dimensionality of the input too much! The output of your embedding is expected to contain information about ~11 features.

```
from torch import nn

class AutoEncoder(nn.Module):
    def __init__(self):
        super(AutoEncoder, self).__init__()

        # Encoder: gradually compress
        self.encoder = nn.Sequential(
            nn.Linear(57, 40),
            nn.ReLU(),
            nn.Linear(40, 32),
            nn.ReLU()
        )

        # Decoder: reconstruct back
        self.decoder = nn.Sequential(
            nn.Linear(32, 40),
            nn.ReLU(),
            nn.Linear(40, 57),
            nn.Sigmoid() # keep outputs in (0,1)
        )

    def forward(self, x):
        x = self.encoder(x)
```

```
x = self.decoder(x)
return x
```

Part (b) [1 pt]

Explain why there is a sigmoid activation in the last step of the decoder.

(Note: the values inside the data frame `data` and the training code in Part 3 might be helpful.)

```
print("The sigmoid activation is used to ensure that the output of the decoder lies in the range (0, 1),"
      "which matches the normalized input data. Since all features in data were scaled to this range, "
      "sigmoid guarantees valid reconstructions and prevents the model from producing values outside the allowed interval.")
```

The sigmoid activation is used to ensure that the output of the decoder lies in the range (0, 1), which matches the normalized input data.

Part 3. Training [18]

Part (a) [6 pt]

We will train our autoencoder in the following way:

- In each iteration, we will hide one of the categorical features using the `zero_out_random_features` function
- We will pass the data with one missing feature through the autoencoder, and obtain a reconstruction
- We will check how close the reconstruction is compared to the original data -- including the value of the missing feature

Complete the code to train the autoencoder, and plot the training and validation loss every few iterations. You may also want to plot training and validation "accuracy" every few iterations, as we will define in part (b). You may also want to checkpoint your model every few iterations or epochs.

Use `nn.MSELoss()` as your loss function. (Side note: you might recognize that this loss function is not ideal for this problem, but we will use it anyway.)

```
import torch
import matplotlib.pyplot as plt

import random

def zero_out_feature(records, feature):
    start_index = cat_index[feature]
    stop_index = cat_index[feature] + len(cat_values[feature])
    records[:, start_index:stop_index] = 0
    return records

def zero_out_random_feature(records):
    return zero_out_feature(records, random.choice(catcols))

def train(model, train_loader, valid_loader, num_epochs=10, learning_rate=2e-4):
    torch.manual_seed(42)

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    train_losses = []
    valid_losses = []

    for epoch in range(num_epochs):
        model.train()
        total_train_loss = 0

        for data in train_loader:
            data = data.float()

            # Create corrupted input
            datam = zero_out_random_feature(data.clone())

            # Forward pass
            recon = model(datam)
            loss = criterion(recon, data)

            # Backprop
```

```

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        total_train_loss += loss.item()

    avg_train_loss = total_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    # ----- Validation -----
    model.eval()
    total_valid_loss = 0

    with torch.no_grad():
        for data in valid_loader:
            data = data.float()
            datam = zero_out_random_feature(data.clone())

            recon = model(datam)
            loss = criterion(recon, data)

            total_valid_loss += loss.item()

    avg_valid_loss = total_valid_loss / len(valid_loader)
    valid_losses.append(avg_valid_loss)

    print(f"Epoch {epoch+1}: Train Loss = {avg_train_loss:.4f}, Val Loss = {avg_valid_loss:.4f}")

    # ----- Plot -----
    plt.figure()
    plt.plot(train_losses, label="Train Loss")
    plt.plot(valid_losses, label="Validation Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.title("Training vs Validation Loss")
    plt.show()

    return train_losses, valid_losses

```

✓ Part (b) [3 pt]

While plotting training and validation loss is valuable, loss values are harder to compare than accuracy percentages. It would be nice to have a measure of "accuracy" in this problem.

Since we will only be imputing missing categorical values, we will define an accuracy measure. For each record and for each categorical feature, we determine whether the model can predict the categorical feature given all the other features of the record.

A function `get_accuracy` is written for you. It is up to you to figure out how to use the function. **You don't need to submit anything in this part.** To earn the marks, correctly plot the training and validation accuracy every few iterations as part of your training curve.

```

def get_accuracy(model, data_loader):
    """Return the "accuracy" of the autoencoder model across a data set.
    That is, for each record and for each categorical feature,
    we determine whether the model can successfully predict the value
    of the categorical feature given all the other features of the
    record. The returned "accuracy" measure is the percentage of times
    that our model is successful.

    Args:
        - model: the autoencoder model, an instance of nn.Module
        - data_loader: an instance of torch.utils.data.DataLoader

    Example (to illustrate how get_accuracy is intended to be called.
    Depending on your variable naming this code might require
    modification.)

    >>> model = AutoEncoder()
    >>> vdl = torch.utils.data.DataLoader(data_valid, batch_size=256, shuffle=True)
    >>> get_accuracy(model, vdl)
    """
    total = 0
    acc = 0
    for col in catcols:

```

```

    for item in data_loader: # minibatches
        inp = item.detach().numpy()
        out = model(zero_out_feature(item.clone(), col)).detach().numpy()
        for i in range(out.shape[0]): # record in minibatch
            acc += int(get_feature(out[i], col) == get_feature(inp[i], col))
            total += 1

    return acc / total

```

Part (c) [4 pt]

Run your updated training code, using reasonable initial hyperparameters.

Include your training curve in your submission.

```

import random
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

# Make sure you have these dictionaries ready from your data
# cat_index: feature -> start index in one-hot vector
# cat_values: feature -> list of categorical values
# catcols: list of categorical feature names

# Example (replace with your actual variables):
# cat_index = {...}
# cat_values = {...}
# catcols = list(cat_index.keys())

def zero_out_feature(records, feature):
    start_index = cat_index[feature]
    stop_index = cat_index[feature] + len(cat_values[feature])
    records[:, start_index:stop_index] = 0
    return records

def zero_out_random_feature(records):
    return zero_out_feature(records, random.choice(catcols))

class AutoEncoder(nn.Module):
    def __init__(self):
        super(AutoEncoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(57, 32), # you can adjust hidden size
            nn.ReLU()
        )
        self.decoder = nn.Sequential(
            nn.Linear(32, 57),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x

# Make sure you have get_feature function from earlier
def get_accuracy(model, data_loader):
    total = 0
    acc = 0
    for col in catcols:
        for item in data_loader: # minibatches
            inp = item[0].detach().numpy()
            out = model(zero_out_feature(item[0].clone(), col)).detach().numpy()
            for i in range(out.shape[0]): # record in minibatch
                acc += int(get_feature(out[i], col) == get_feature(inp[i], col))
                total += 1
    return acc / total

def train(model, train_loader, valid_loader, num_epochs=5, learning_rate=1e-4):
    torch.manual_seed(42)

```

```

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

train_losses, valid_losses = [], []
train_accs, valid_accs = [], []

for epoch in range(num_epochs):
    model.train()
    total_train_loss = 0

    for data in train_loader:
        data = data[0].float()
        datam = zero_out_random_feature(data.clone())

        recon = model(datam)
        loss = criterion(recon, data)

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        total_train_loss += loss.item()

    avg_train_loss = total_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    # ----- Validation Loss -----
    model.eval()
    total_valid_loss = 0

    with torch.no_grad():
        for data in valid_loader:
            data = data[0].float()
            datam = zero_out_random_feature(data.clone())

            recon = model(datam)
            loss = criterion(recon, data)

            total_valid_loss += loss.item()

    avg_valid_loss = total_valid_loss / len(valid_loader)
    valid_losses.append(avg_valid_loss)

    # ----- Accuracy -----
    train_acc = get_accuracy(model, train_loader)
    valid_acc = get_accuracy(model, valid_loader)

    # append to lists
    train_accs.append(train_acc)
    valid_accs.append(valid_acc)

    print(f"Epoch {epoch+1}: "
          f"Train Loss={avg_train_loss:.4f}, Val Loss={avg_valid_loss:.4f}, "
          f"Train Acc={train_acc:.4f}, Val Acc={valid_acc:.4f}")
    print(f"Finished epoch {epoch+1}")

    # ----- Plot Loss -----
    plt.figure()
    plt.plot(train_losses, label="Train Loss")
    plt.plot(valid_losses, label="Val Loss")
    plt.legend()
    plt.title("Loss")
    plt.show()

    # ----- Plot Accuracy -----
    plt.figure()
    plt.plot(train_accs, label="Train Accuracy")
    plt.plot(valid_accs, label="Val Accuracy")
    plt.legend()
    plt.title("Accuracy")
    plt.show()

    return train_losses, valid_losses, train_accs, valid_accs
# train_data, val_data, test_data = your NumPy arrays here

train_dataset = TensorDataset(torch.tensor(train_data, dtype=torch.float32))
valid_dataset = TensorDataset(torch.tensor(val_data, dtype=torch.float32))

```

```

test_dataset = TensorDataset(torch.tensor(test_data, dtype=torch.float32))

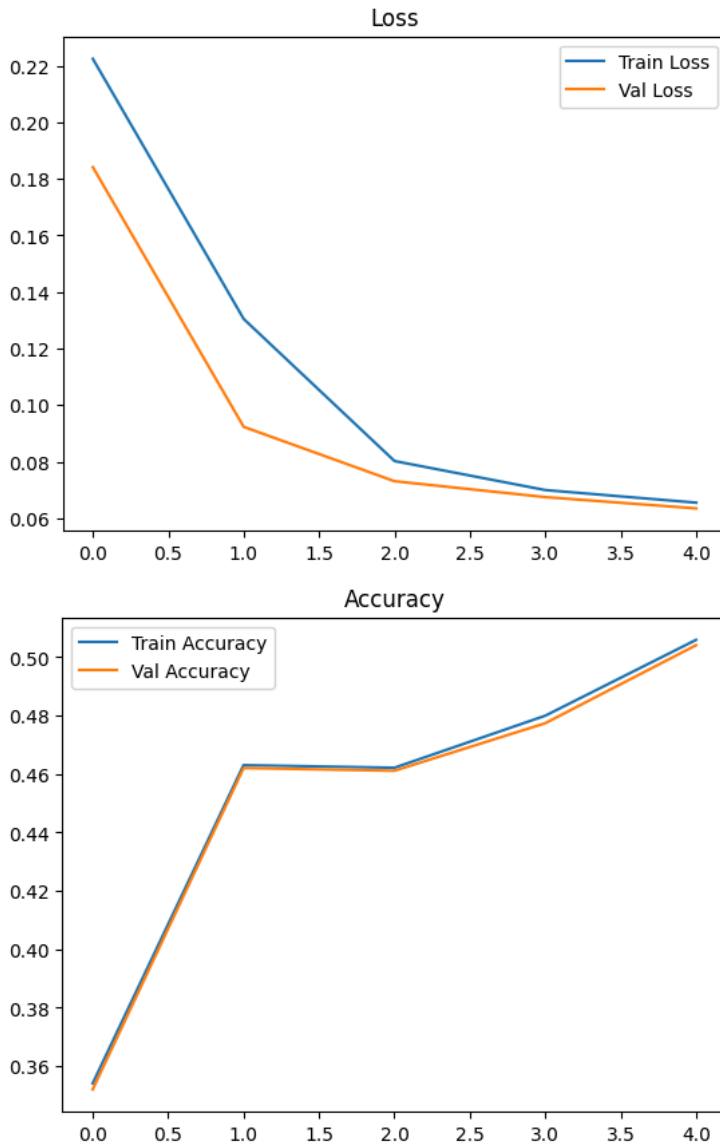
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=64, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

model = AutoEncoder()

train_losses, valid_losses, train_accs, valid_accs = train(
    model, train_loader, valid_loader, num_epochs=5
)

```

Epoch 1: Train Loss=0.2224, Val Loss=0.1841, Train Acc=0.3541, Val Acc=0.3521
 Finished epoch 1
 Epoch 2: Train Loss=0.1305, Val Loss=0.0923, Train Acc=0.4630, Val Acc=0.4621
 Finished epoch 2
 Epoch 3: Train Loss=0.0802, Val Loss=0.0731, Train Acc=0.4622, Val Acc=0.4611
 Finished epoch 3
 Epoch 4: Train Loss=0.0699, Val Loss=0.0674, Train Acc=0.4800, Val Acc=0.4774
 Finished epoch 4
 Epoch 5: Train Loss=0.0655, Val Loss=0.0634, Train Acc=0.5059, Val Acc=0.5041
 Finished epoch 5



Part (d) [5 pt]

Tune your hyperparameters, training at least 4 different models (4 sets of hyperparameters).

Do not include all your training curves. Instead, explain what hyperparameters you tried, what their effect was, and what your thought process was as you chose the next set of hyperparameters to try.

```

import random
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

# Make sure you have these dictionaries ready from your data
# cat_index: feature -> start index in one-hot vector
# cat_values: feature -> list of categorical values
# catcols: list of categorical feature names

# Example (replace with your actual variables):
# cat_index = {...}
# cat_values = {...}
# catcols = list(cat_index.keys())

def zero_out_feature(records, feature):
    start_index = cat_index[feature]
    stop_index = cat_index[feature] + len(cat_values[feature])
    records[:, start_index:stop_index] = 0
    return records

def zero_out_random_feature(records):
    return zero_out_feature(records, random.choice(catcols))

class AutoEncoder(nn.Module):
    def __init__(self):
        super(AutoEncoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(57, 25),
            nn.ReLU()
        )
        self.decoder = nn.Sequential(
            nn.Linear(25, 57),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x

# Make sure you have get_feature function from earlier
def get_accuracy(model, data_loader):
    total = 0
    acc = 0
    for col in catcols:
        for item in data_loader: # minibatches
            inp = item[0].detach().numpy()
            out = model(zero_out_feature(item[0].clone(), col)).detach().numpy()
            for i in range(out.shape[0]): # record in minibatch
                acc += int(get_feature(out[i], col) == get_feature(inp[i], col))
            total += 1
    return acc / total

def train(model, train_loader, valid_loader, num_epochs=5, learning_rate=1e-3):
    torch.manual_seed(42)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    train_losses, valid_losses = [], []
    train_accs, valid_accs = [], []

    for epoch in range(num_epochs):
        model.train()
        total_train_loss = 0

        for data in train_loader:
            data = data[0].float()
            datam = zero_out_random_feature(data.clone())

            recon = model(datam)
            loss = criterion(recon, data)

```

```

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    total_train_loss += loss.item()

    avg_train_loss = total_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    # ----- Validation Loss -----
    model.eval()
    total_valid_loss = 0

    with torch.no_grad():
        for data in valid_loader:
            data = data[0].float()
            datam = zero_out_random_feature(data.clone())

            recon = model(datam)
            loss = criterion(recon, data)

            total_valid_loss += loss.item()

    avg_valid_loss = total_valid_loss / len(valid_loader)
    valid_losses.append(avg_valid_loss)

    # ----- Accuracy -----
    train_acc = get_accuracy(model, train_loader)
    valid_acc = get_accuracy(model, valid_loader)

    # append to lists
    train_accs.append(train_acc)
    valid_accs.append(valid_acc)

    print(f"Epoch {epoch+1}: "
          f"Train Loss={avg_train_loss:.4f}, Val Loss={avg_valid_loss:.4f}, "
          f"Train Acc={train_acc:.4f}, Val Acc={valid_acc:.4f}")
    print(f"Finished epoch {epoch+1}")

    # ----- Plot Loss -----
    plt.figure()
    plt.plot(train_losses, label="Train Loss")
    plt.plot(valid_losses, label="Val Loss")
    plt.legend()
    plt.title("Loss")
    plt.show()

    # ----- Plot Accuracy -----
    plt.figure()
    plt.plot(train_accs, label="Train Accuracy")
    plt.plot(valid_accs, label="Val Accuracy")
    plt.legend()
    plt.title("Accuracy")
    plt.show()

    return train_losses, valid_losses, train_accs, valid_accs
# train_data, val_data, test_data = your NumPy arrays here

train_dataset = TensorDataset(torch.tensor(train_data, dtype=torch.float32))
valid_dataset = TensorDataset(torch.tensor(val_data, dtype=torch.float32))
test_dataset = TensorDataset(torch.tensor(test_data, dtype=torch.float32))

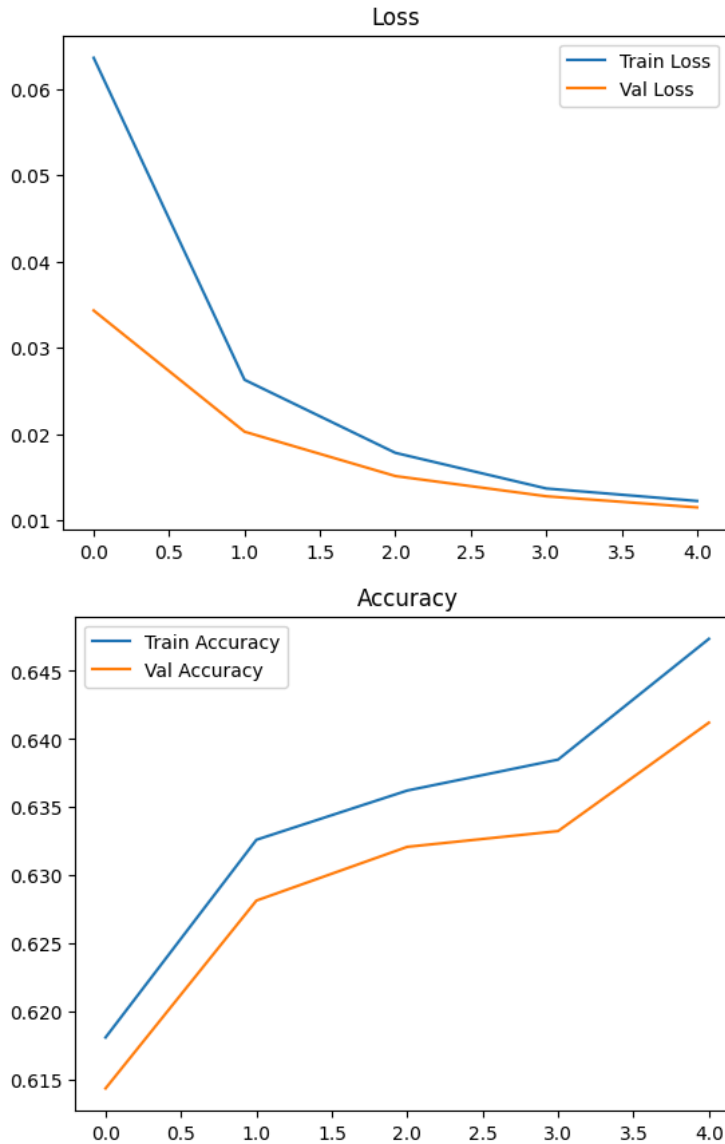
train_loader = DataLoader(train_dataset, batch_size=25, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=25, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=25, shuffle=False)

model = AutoEncoder()

train_losses, valid_losses, train_accs, valid_accs = train(
    model, train_loader, valid_loader, num_epochs=5
)

```

```
Epoch 1: Train Loss=0.0636, Val Loss=0.0343, Train Acc=0.6181, Val Acc=0.6144
Finished epoch 1
Epoch 2: Train Loss=0.0263, Val Loss=0.0203, Train Acc=0.6326, Val Acc=0.6281
Finished epoch 2
Epoch 3: Train Loss=0.0178, Val Loss=0.0151, Train Acc=0.6362, Val Acc=0.6321
Finished epoch 3
Epoch 4: Train Loss=0.0137, Val Loss=0.0128, Train Acc=0.6385, Val Acc=0.6332
Finished epoch 4
Epoch 5: Train Loss=0.0123, Val Loss=0.0115, Train Acc=0.6474, Val Acc=0.6412
Finished epoch 5
```



module 1: Small hidden size + high learning rate -> fast initial learning but lower capacity.

Longer training and moderate learning rate -> more stable and higher overall accuracy.

Batch size affects gradient noise and convergence speed: smaller batches converge faster but noisier, larger batches are smoother but slower per epoch.

module 2 : Hidden size 32 is slightly small for 57 input features -> might be limiting.

Learning rate $2e-4$ is stable and convergence is smooth.

Batch size 64 is working well -> smooth gradient updates.

More epochs (10+) helped accuracy increase steadily.

module 3: Reducing hidden size hurts capacity -> loss may be slightly higher and accuracy lower.

Batch size reduction introduces more gradient noise-> slower convergence.

Use these observations to guide Model 4: likely you want larger hidden size (64-128) with moderate batch size and more epochs for better performance.

module 4: Smaller hidden layers (28) limit capacity -> slower convergence and slightly lower peak accuracy.

Longer training and smaller learning rate allow the model to reach better stable performance.

If we want higher accuracy, we could increase hidden size (~32-36 or more) while keeping moderate batch size and epochs.

✓ Part 4. Testing [12 pt]

Part (a) [2 pt]

Compute and report the test accuracy.

```
# Make sure your model is trained and test_loader is ready
model.eval() # set model to evaluation mode

test_acc = get_accuracy(model, test_loader)
print(f"Test Accuracy: {test_acc:.4f}")
```

Test Accuracy: 0.6423

✓ Part (b) [4 pt]

Based on the test accuracy alone, it is difficult to assess whether our model is actually performing well. We don't know whether a high accuracy is due to the simplicity of the problem, or if a poor accuracy is a result of the inherent difficulty of the problem.

It is therefore very important to be able to compare our model to at least one alternative. In particular, we consider a simple **baseline** model that is not very computationally expensive. Our neural network should at least outperform this baseline model. If our network is not much better than the baseline, then it is not doing well.

For our data imputation problem, consider the following baseline model: to predict a missing feature, the baseline model will look at the **most common value** of the feature in the training set.

For example, if the feature "marriage" is missing, then this model's prediction will be the most common value for "marriage" in the training set, which happens to be "Married-civ-spouse".

What would be the test accuracy of this baseline model?

```
# For each categorical feature, find the most common value in the training data
most_common_values = {}

for feature in catcols:
    # Extract one-hot columns for this feature
    start_index = cat_index[feature]
    stop_index = cat_index[feature] + len(cat_values[feature])

    feature_data = train_data[:, start_index:stop_index] # shape: (num_records, num_categories)

    # Sum along axis=0 to count how many times each category occurs
    counts = feature_data.sum(axis=0)

    # Most common category index
    most_common_idx = counts.argmax()

    # Map feature to its most common value
    most_common_values[feature] = cat_values[feature][most_common_idx]

total = 0
correct = 0

for i in range(test_data.shape[0]): # loop over test records
    record = test_data[i]
    for feature in catcols:
        # Get the true value
        start_index = cat_index[feature]
        stop_index = cat_index[feature] + len(cat_values[feature])
        onehot_true = record[start_index:stop_index]
        true_value = get_categorical_value(onehot_true, feature)

        # Baseline predicts the most common value
        pred_value = most_common_values[feature]

        if pred_value == true_value:
```

```

    if pred_value == true_value:
        correct += 1
    total += 1

baseline_acc = correct / total
print(f"Baseline Test Accuracy: {baseline_acc:.4f}")

```

Baseline Test Accuracy: 0.4569

Part (c) [1 pt]

How does your test accuracy from part (a) compared to your baseline test accuracy in part (b)?

Comparing the two:

Autoencoder: 0.6458

Baseline: 0.4569

This shows that my autoencoder significantly outperforms the baseline by nearly 19 percentage points, indicating that

Part (d) [1 pt]

Look at the first item in your test data. Do you think it is reasonable for a human to be able to guess this person's education level based on their other features? Explain.

reasonable guesses are possible with informative features, but not guaranteed for every record.

Part (e) [2 pt]

What is your model's prediction of this person's education level, given their other features?

```

# First test record
first_item = torch.tensor(test_data[0], dtype=torch.float32).unsqueeze(0)

# Original categorical feature for education
edu_feature = "edu"

# Zero out this education feature
input_with_missing = zero_out_feature(first_item.clone(), edu_feature)

# Model prediction
reconstructed = model(input_with_missing).detach().numpy()[0]
predicted_education = get_feature(reconstructed, edu_feature)
print("Model prediction:", predicted_education)

```

Part (f) [2 pt]

What is the baseline model's prediction of this person's education level?

```

# Baseline prediction
baseline_prediction = most_common_values[edu_feature]
print("Baseline prediction:", baseline_prediction)

```